```
/*
==============================================================================
 * STM32 Smoke Detector with Alarm    (matches breadboard photo setup)
 *
==============================================================================
 * Hardware seen in your build:
 *     - MCU         : STM32 "Blue Pill" style minimal dev board (STM32F103C8T6)
 *     - Sensor    : Bare smoke/gas sensor element (analog resistive output,
 *                     MQ-2/MQ-135 type sensing head) - wired directly, no
 *                     breakout board
 *     - Buzzer    : Active buzzer (black 2-pin component)
 *     - LED         : Single red LED (alarm indicator)
 *
 * Pin Mapping (CHANGE THESE to match your actual jumper wires -
 * I assumed a typical layout since the silkscreen wasn't fully readable):
 *
 *     Sensor analog out    -> PA0     (ADC1_IN0)
 *     Buzzer (+)               -> PB0
 *     Red LED (+, via resistor) -> PB1
 *
 * NOTE ON THE SENSOR:
 *     A bare sensor element (just the metal-mesh "can") only gives you a
 *     raw resistance that changes with gas concentration - it needs a load
 *     resistor (RL, typically 5-20k) between the sensor output pin and GND
 *     to form a voltage divider so the STM32 ADC can read a usable voltage.
 *     If your sensor already has leads going to the breadboard rows as
 *     shown, check that one of those rows also connects to a resistor down
 *     to GND - that junction is what should go to PA0. If there's no load
 *     resistor in the circuit, add a 10k resistor from the sensor's signal
 *     pin to GND.
 *
 * Logic:
 *     - Reads analog smoke concentration via ADC every 500 ms.
 *     - Above SMOKE_THRESHOLD -> buzzer beeps, red LED blinks (alarm).
 *     - Below threshold            -> buzzer off, LED off (safe/idle).
 *     - UART1 prints live ADC readings for calibration/debugging.
 *
 * Toolchain: STM32CubeIDE / STM32Cube HAL library
 *
==============================================================================
*/

#include "main.h"
#include <stdio.h>
#include <string.h>

/* ---------------- Configuration ---------------- */
#define SMOKE_THRESHOLD        1500        // 12-bit ADC (0-4095). Calibrate using
                                           // the UART readings: note the "clean
                                           // air" value, then set threshold a bit
                                           // above it after testing with smoke.
#define ALARM_BEEP_DELAY_MS 200          // Buzzer/LED blink interval during alarm

/* ---------------- Handles ---------------- */
ADC_HandleTypeDef hadc1;
UART_HandleTypeDef huart1;

/* ---------------- Function Prototypes ---------------- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
```

```
static void MX_ADC1_Init(void);
static void MX_USART1_UART_Init(void);
uint32_t Read_Smoke_Sensor(void);
void Alarm_ON(void);
void Alarm_OFF(void);
void UART_Print(char *msg);

/* ============================================================
 * MAIN
 * ============================================================ */
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_USART1_UART_Init();

    char buffer[64];
    uint32_t smokeLevel = 0;

    UART_Print("Smoke Detector Initialized. Warming up sensor...\r\n");
    HAL_Delay(20000); // Gas sensors need ~20-60s warm-up for stable readings

    UART_Print("Sensor ready. Monitoring...\r\n");

    while (1)
    {
        smokeLevel = Read_Smoke_Sensor();

        sprintf(buffer, "Smoke Level: %lu\r\n", smokeLevel);
        UART_Print(buffer);

        if (smokeLevel > SMOKE_THRESHOLD)
        {
            UART_Print("!! ALERT: Smoke Detected !!\r\n");
            Alarm_ON();     // beeps + blinks once, then loop re-checks
        }
        else
        {
            Alarm_OFF();
            HAL_Delay(500); // normal sampling interval
        }
    }
}

/* ============================================================
 * Read smoke sensor value from ADC (PA0)
 * ============================================================ */
uint32_t Read_Smoke_Sensor(void)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    uint32_t value = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return value;
}

/* ============================================================
```

```
 * Alarm Trigger: Buzzer + Red LED, beeping/blinking pattern
 * ============================================================== */
void Alarm_ON(void)
{
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);     // Buzzer ON
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);     // LED ON
    HAL_Delay(ALARM_BEEP_DELAY_MS);

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Buzzer OFF
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET); // LED OFF
    HAL_Delay(ALARM_BEEP_DELAY_MS);
}

/* ==============================================================
 * Normal / Safe State
 * ============================================================== */
void Alarm_OFF(void)
{
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Buzzer OFF
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET); // LED OFF
}

/* ==============================================================
 * UART Debug Print
 * ============================================================== */
void UART_Print(char *msg)
{
    HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
}

/* ==============================================================
 * GPIO Initialization
 * ============================================================== */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    // Buzzer (PB0) and LED (PB1) as digital outputs
    GPIO_InitStruct.Pin = GPIO_PIN_0 | GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    // Sensor analog input (PA0)
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    // Start in safe state
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
}

/* ==============================================================
 * ADC1 Initialization (12-bit, channel 0 -> PA0)
 * ============================================================== */
```

```
static void MX_ADC1_Init(void)
{
    ADC_ChannelConfTypeDef sConfig = {0};

    __HAL_RCC_ADC1_CLK_ENABLE();

    hadc1.Instance = ADC1;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    HAL_ADC_Init(&hadc1);

    sConfig.Channel = ADC_CHANNEL_0;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}

/* ============================================================
 * USART1 Initialization (Debug output, 115200 baud)
 * ============================================================ */
static void MX_USART1_UART_Init(void)
{
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    HAL_UART_Init(&huart1);
}

/* ============================================================
 * System Clock Configuration (72 MHz using HSE, typical Blue Pill setup)
 * ============================================================ */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);

    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
                                | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
```

```
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2);
}
```